

MEXI-BANCO

El objetivo principal de esta práctica es desplegar y probar localmente la aplicación de "Mexi Banco" utilizando Docker Compose, con la finalidad de comprender su estructura.

Paso 1. Descarga de Repo

Primero debemos tener descargada la aplicación de escritorio de Postman, descargar la colección correspondiente, clonar el repositorio mediante el comando git clone --branch v05.1 <https://github.com/OscarRuiz21/mexi-banco.git>

```
D:\>git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 173.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
warning: refs/tags/v05.1 cd46b8b91a09070f8262c3b8acfb17ca2405cda is not a commit!
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false
```

Paso 2 · Docker Compose

Una vez clonado el repositorio, procederemos a levantar el entorno con el comando docker compose up --build -d, comprobando que esté correctamente operativo con docker compose ps, sabiendo que se levantó de forma exitosa porque su estado será *healthy*.

```
D:\mexi-banco>docker compose up --build -d
[*] Building 3 is (20/20) FINISHED
=> [internal] load local base definitions
=> => reading from stdin 475B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 515B
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
=> [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk-alpine
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io
=> [internal] load dockerignore
=> => transferring context: 146B
=> [builder 1/8] FROM docker.io/library/eclipse-temurin:21-jdk-alpine@sha256:0bfc69a4758a86710e5c474832d28408a8bd08874766f9e8b1642ac2fd293159
=> => resolve docker.io/library/eclipse-temurin:21-jdk-alpine@sha256:0bfc69a4758a86710e5c474832d28408a8bd08874766f9e8b1642ac2fd293159
=> [internal] load build context
=> => transferring context: 3.38kB
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e3302a7141ce665ca3ea85e8b5cd648eafbc3c0c90dd79dd537684e4555
=> => resolve docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e3302a7141ce665ca3ea85e8b5cd648eafbc3c0c90dd79dd537684e4555
=> CACHED [stage-1 2/3] WORKDIR /app
=> CACHED [builder 2/8] WORKDIR /app
=> CACHED [builder 3/8] COPY pom.xml .
=> CACHED [builder 4/8] COPY mvnw .
=> CACHED [builder 5/8] COPY .mvn .mvn
=> CACHED [builder 6/8] RUN apk add dos2unix && dos2unix ./mvnw && chmod +x ./mvnw && ./mvnw dependency:go-offline -B
=> CACHED [builder 7/8] COPY src ./src
=> CACHED [builder 8/8] RUN ./mvnw clean package -DskipTests
=> CACHED [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:8c1d0b8863c8cfb3113924db864043aa0fa1f361ed840beb2584cbe687076d53
=> => exporting config sha256:57043f2f33716e8be6a4b1518c1a36b799b6ca2711ca8d4d9b152616127d6c03
=> => exporting attestation manifest sha256:8719a56d6ba40979274c5fcd8a0d150a11c8e0985a5b3404cda3d189e67dd32
=> => exporting manifest list sha256:2c955b7af7f984f9451bc305e8f5806b08d08397941a28d8b6dc316f04796bd
=> => naming to docker.io/library/mexi-banco-app:latest
=> => unpacking to docker.io/library/mexi-banco-app:latest
=> => resolving provenance for metadata file
[*] up 3/3
✔image mexi-banco-app Built 1.7s
✔Container mexi-banco-db-1 Healthy 1.6s
✔Container mexi-banco-app-1 Started 2.2s
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
mexi-banco-app-1	sha256:b9dd56363fb6e23ecb45e9337b8bf0b49200858ac3efae18c38672dfb6dbd0e	"java -jar app.jar"	app	3 minutes ago	Up 3 minutes (healthy)	0.0.0.0
mexi-banco-app-1	8080->8080/tcp, [::]:8080->8080/tcp					
mexi-banco-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	3 minutes ago	Up 3 minutes (healthy)	5432/tcp

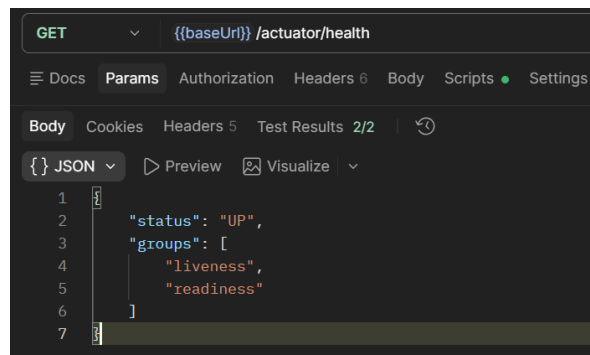
D:\mexi-banco>

Paso 3 · Pruebas con Postman

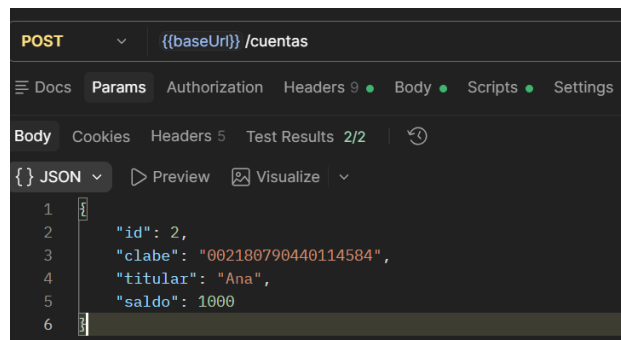
Para validar el funcionamiento del sistema, el siguiente paso consiste en verificar las rutas y operaciones de la API mediante Postman, utilizando para ello la colección previamente obtenida para ejecutar de manera secuencial cada uno de los *endpoints* disponibles.

- **Arriba y respondiendo.**

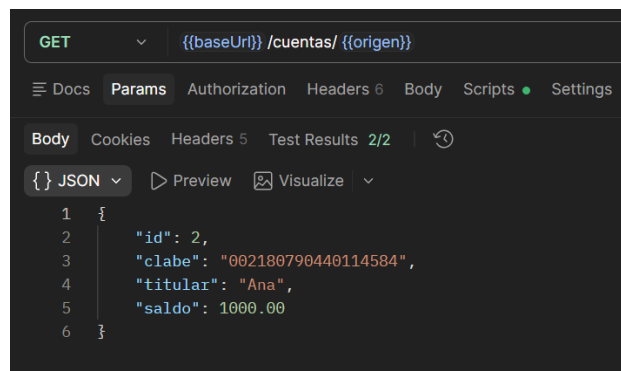
Probamos el estado del sistema.



Creamos una cuenta para Ana y le asignamos un saldo.

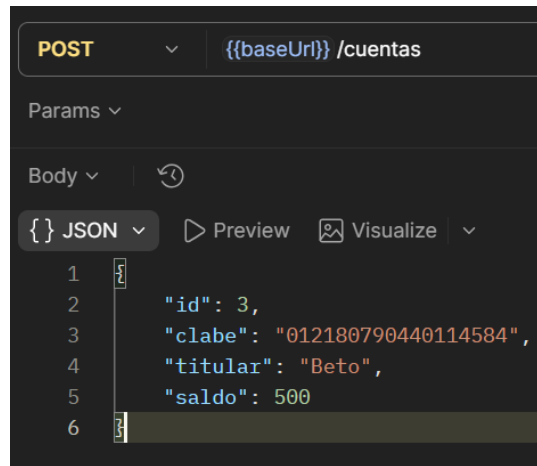


Consultaremos la cuenta que asignamos con GET.



- **Transferencia Interna**

Definimos una cuenta de destino para la transferencia, Beto con saldo de 500

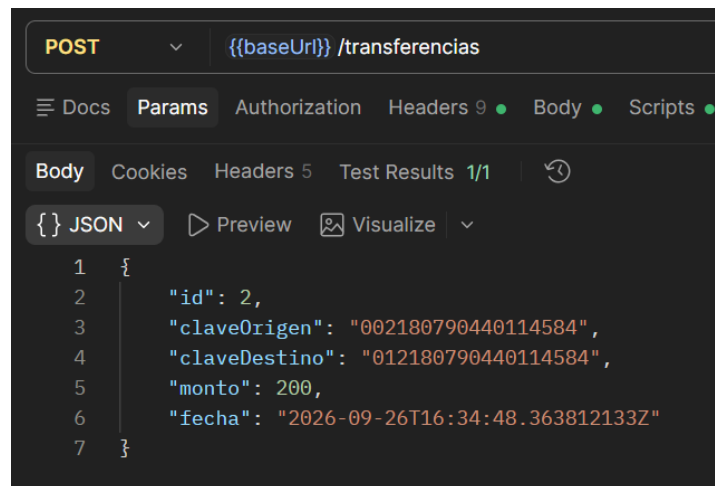


```
POST {{baseUrl}}/cuentas

Params

Body
  {} JSON Preview Visualize
  1 {
  2   "id": 3,
  3   "clabe": "012180790440114584",
  4   "titular": "Beto",
  5   "saldo": 500
  6 }
```

Se manda de Ana a Beto con un monto de 200.

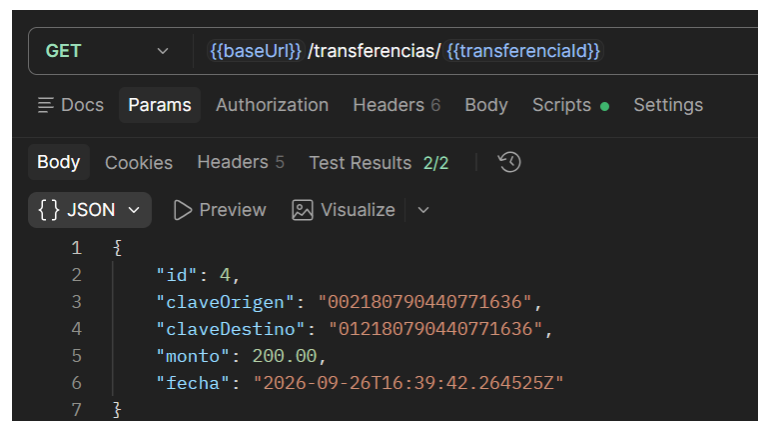


```
POST {{baseUrl}}/transferencias

Docs Params Authorization Headers 9 Body Scripts

Body Cookies Headers 5 Test Results 1/1
  {} JSON Preview Visualize
  1 {
  2   "id": 2,
  3   "claveOrigen": "002180790440114584",
  4   "claveDestino": "012180790440114584",
  5   "monto": 200,
  6   "fecha": "2026-09-26T16:34:48.363812133Z"
  7 }
```

Consultamos la transferencia que se realizó en cuentas

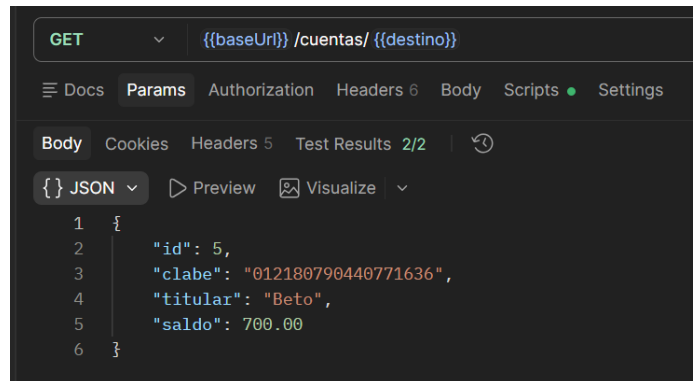


```
GET {{baseUrl}}/transferencias/{{transferenciald}}

Docs Params Authorization Headers 6 Body Scripts Settings

Body Cookies Headers 5 Test Results 2/2
  {} JSON Preview Visualize
  1 {
  2   "id": 4,
  3   "claveOrigen": "002180790440771636",
  4   "claveDestino": "012180790440771636",
  5   "monto": 200.00,
  6   "fecha": "2026-09-26T16:39:42.264525Z"
  7 }
```

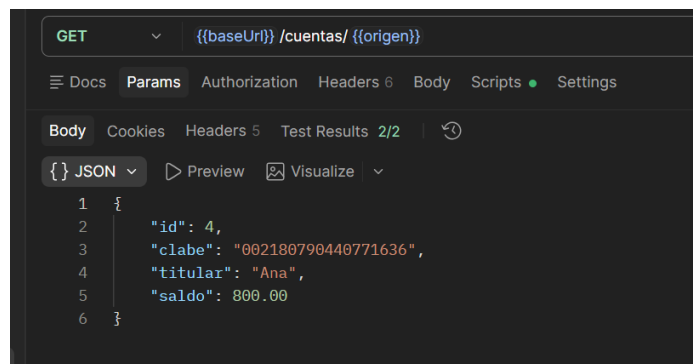
Debido a la transacción, Ana termino con un saldo 800 y Beto con 700.



```
GET {{baseUrl}}/cuentas/{{destino}}

Body
  Cookies
  Headers 5
  Test Results 2/2

JSON
1 {
2   "id": 5,
3   "clabe": "012180790440771636",
4   "titular": "Beto",
5   "saldo": 700.00
6 }
```



```
GET {{baseUrl}}/cuentas/{{origen}}

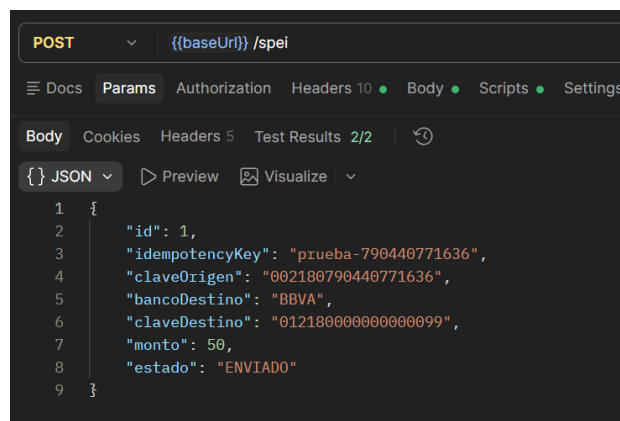
Body
  Cookies
  Headers 5
  Test Results 2/2

JSON
1 {
2   "id": 4,
3   "clabe": "002180790440771636",
4   "titular": "Ana",
5   "saldo": 800.00
6 }
```

- **Idempotencia.**

Comprobaremos la propiedad de idempotencia. Este principio garantiza que ejecutar una misma operación varias veces genera el mismo resultado que hacerlo una sola vez, previniendo cargos duplicados en las cuentas.

Ejecutaremos una transferencia inicial con el identificador único 1, descontando 50 unidades de la cuenta origen, cuyo saldo inicial era de 800, dejándola en un total de 750.

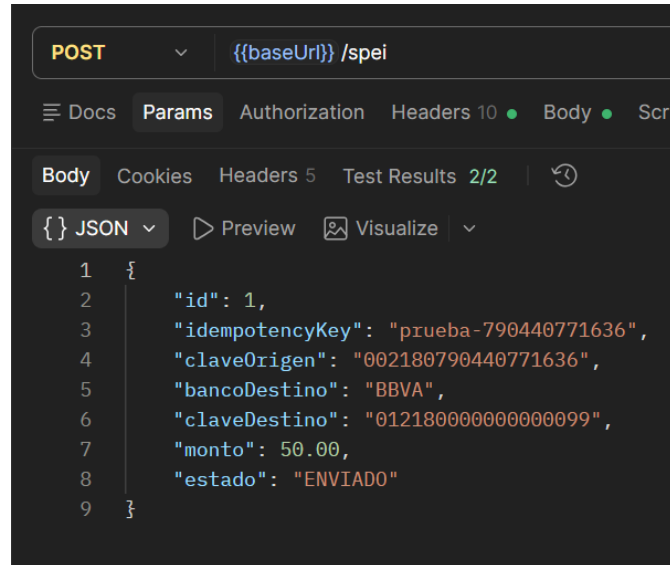


```
POST {{baseUrl}}/spei

Body
  Cookies
  Headers 5
  Test Results 2/2

JSON
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-790440771636",
4   "clabeOrigen": "002180790440771636",
5   "bancoDestino": "BBVA",
6   "clabeDestino": "012180000000000099",
7   "monto": 50,
8   "estado": "ENVIADO"
9 }
```

Posteriormente, simularemos un reenvío enviando la misma solicitud con exactamente el mismo identificador (id: 1).



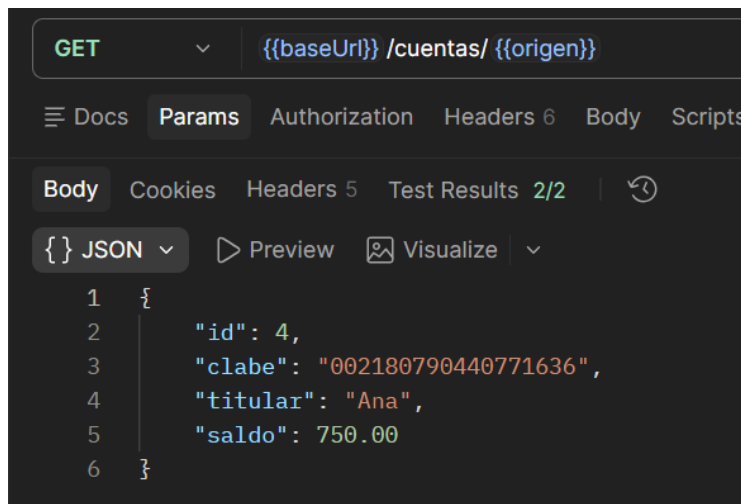
```
POST {{baseUrl}}/spei

Body
  Cookies
  Headers 5
  Test Results 2/2

  {} JSON Preview Visualize

1  {
2    "id": 1,
3    "idempotencyKey": "prueba-790440771636",
4    "claveOrigen": "002180790440771636",
5    "bancoDestino": "BBVA",
6    "claveDestino": "012180000000000099",
7    "monto": 50.00,
8    "estado": "ENVIADO"
9  }
```

Al consultar nuevamente el estado de la cuenta origen, el saldo deberá permanecer intacto en 750, confirmando que el sistema reconoce la petición duplicada y evita procesarla de nuevo.



```
GET {{baseUrl}}/cuentas/{{origen}}

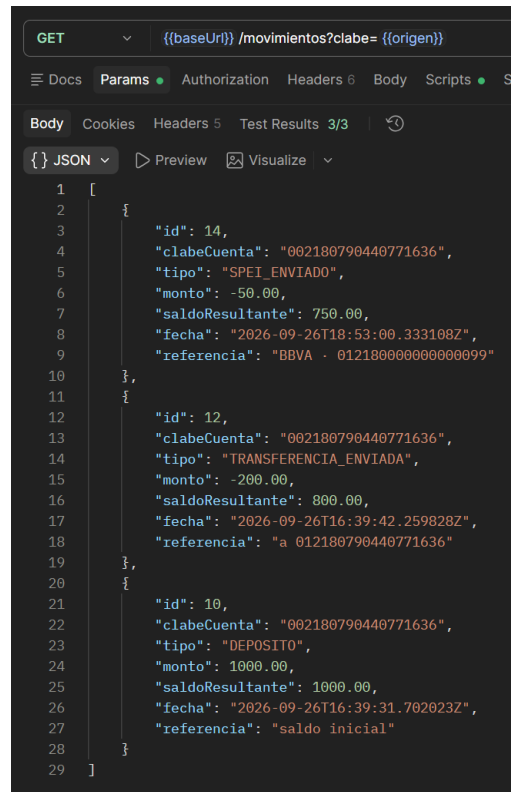
Body
  Cookies
  Headers 5
  Test Results 2/2

  {} JSON Preview Visualize

1  {
2    "id": 4,
3    "clabe": "002180790440771636",
4    "titular": "Ana",
5    "saldo": 750.00
6  }
```

- **Monolito**

Podemos revisar todo los movimientos de las cuentas, tanto las transacciones y orígenes de cuenta.

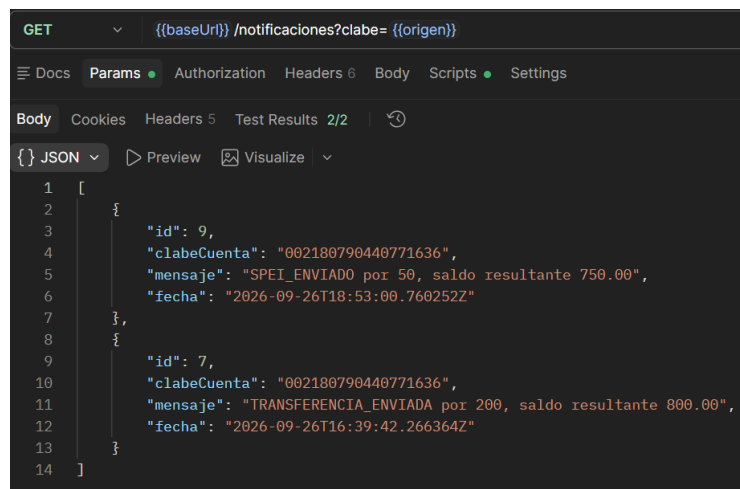


```
GET {{baseUrl}} /movimientos?clabe= {{origen}}

Body
Cookies
Headers 5
Test Results 3/3
Visualize

[
  {
    "id": 14,
    "clabeCuenta": "002180790440771636",
    "tipo": "SPEI_ENVIADO",
    "monto": -50.00,
    "saldoResultante": 750.00,
    "fecha": "2026-09-26T18:53:00.333108Z",
    "referencia": "BBVA - 012180000000000099"
  },
  {
    "id": 12,
    "clabeCuenta": "002180790440771636",
    "tipo": "TRANSFERENCIA_ENVIADA",
    "monto": -200.00,
    "saldoResultante": 800.00,
    "fecha": "2026-09-26T16:39:42.259828Z",
    "referencia": "a 012180790440771636"
  },
  {
    "id": 10,
    "clabeCuenta": "002180790440771636",
    "tipo": "DEPOSITO",
    "monto": 1000.00,
    "saldoResultante": 1000.00,
    "fecha": "2026-09-26T16:39:31.702023Z",
    "referencia": "saldo inicial"
  }
]
```

Consultamos las notificaciones, donde están nuestros movimientos.



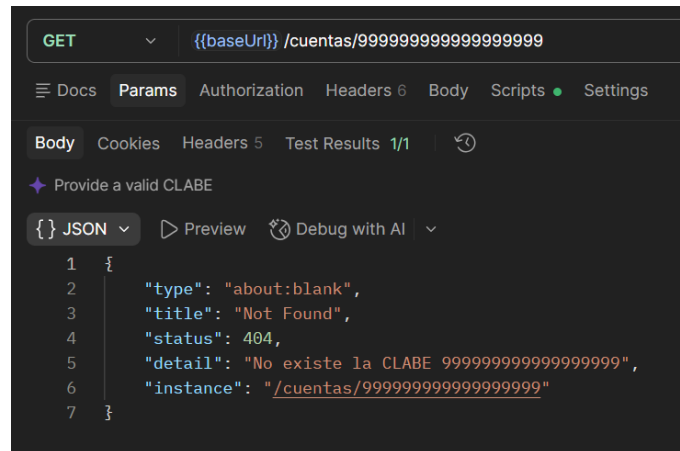
```
GET {{baseUrl}} /notificaciones?clabe= {{origen}}

Body
Cookies
Headers 5
Test Results 2/2
Visualize

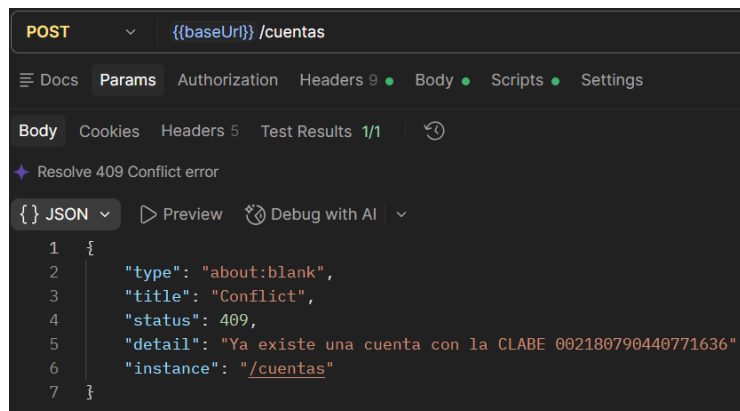
[
  {
    "id": 9,
    "clabeCuenta": "002180790440771636",
    "mensaje": "SPEI_ENVIADO por 50, saldo resultante 750.00",
    "fecha": "2026-09-26T18:53:00.760252Z"
  },
  {
    "id": 7,
    "clabeCuenta": "002180790440771636",
    "mensaje": "TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00",
    "fecha": "2026-09-26T16:39:42.266364Z"
  }
]
```

- **Errores**

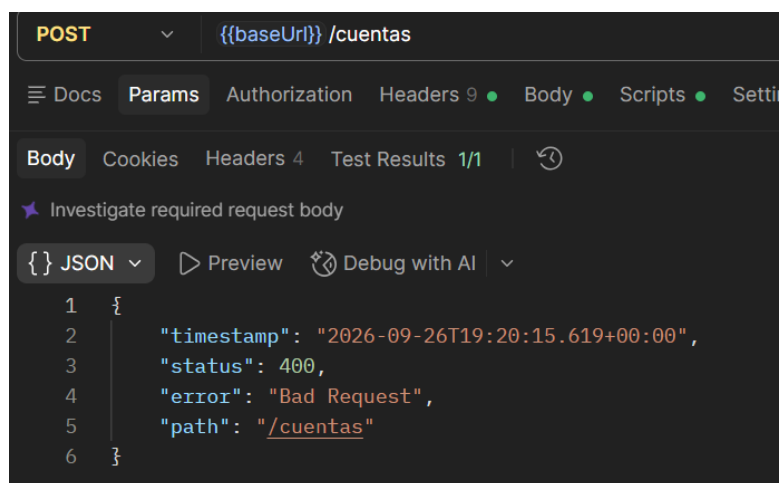
1. No se encontró la cuenta 99999...



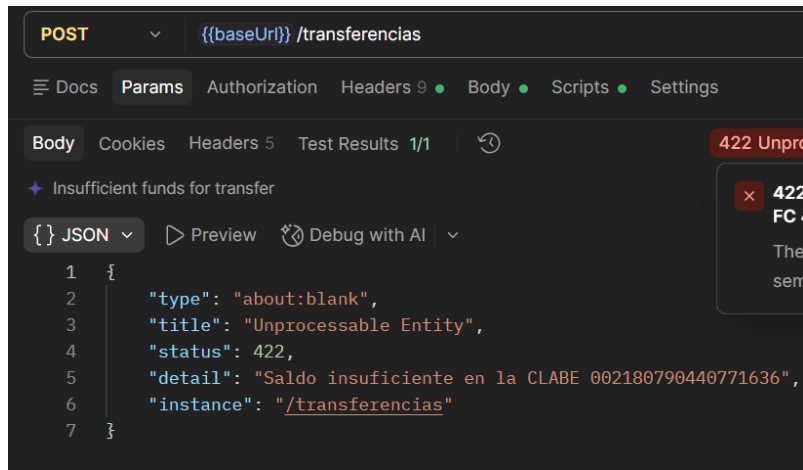
- ## 2. Cuando ya hay un clave existente



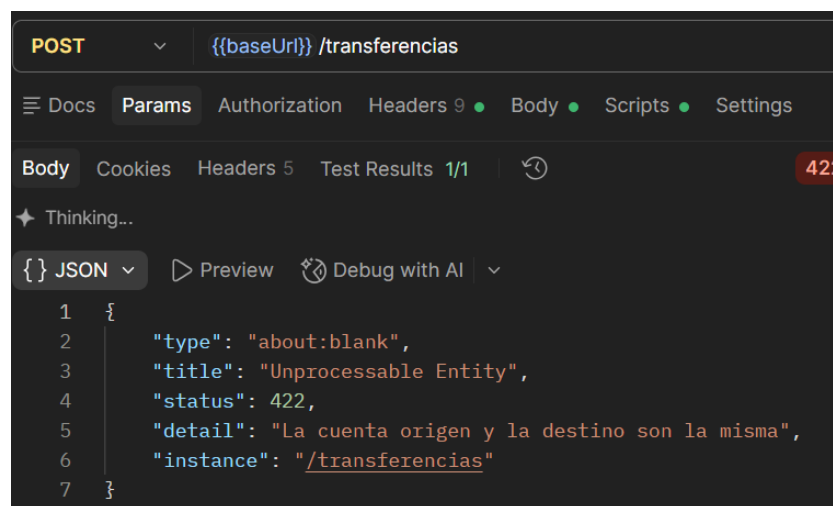
- ### 3. Cuando una solicitud sin cuerpo



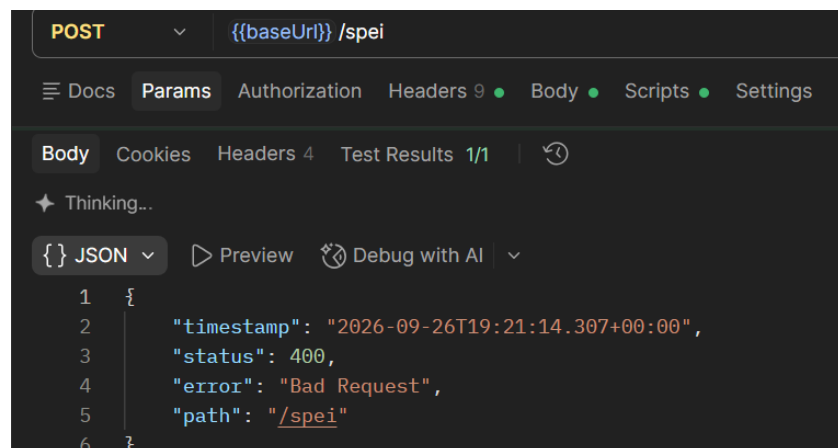
4. Cuando hay saldo insuficiente



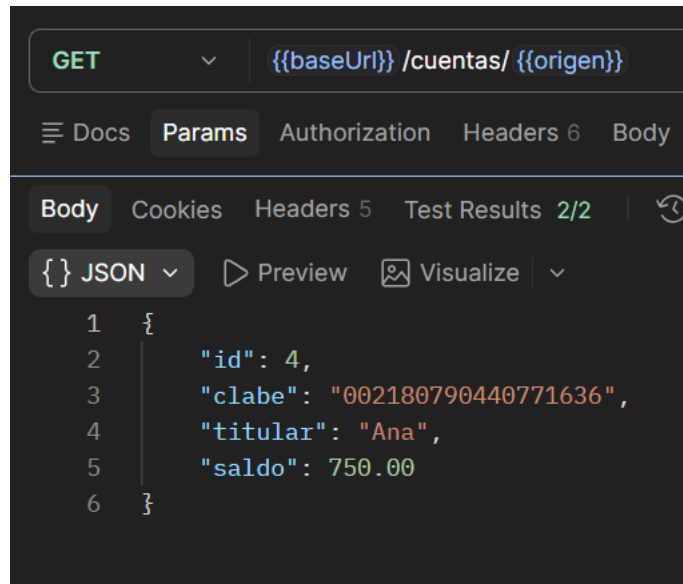
5. Cuando la cuenta destino y origen s la misma



6. SPEI sin idempotencia.

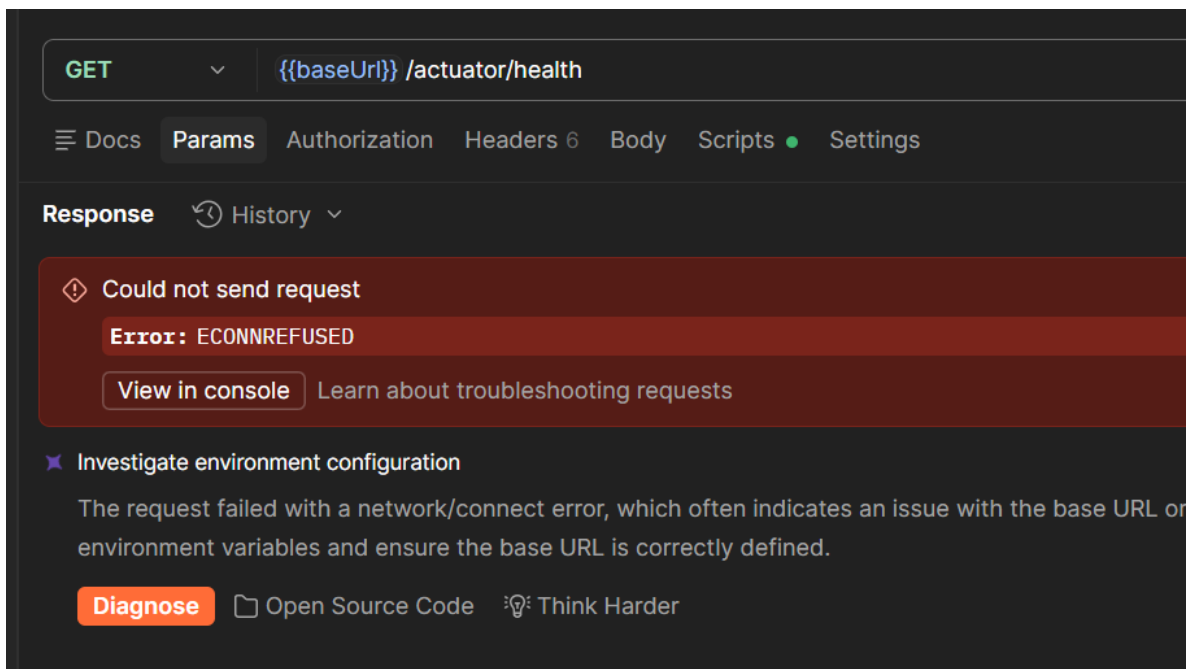


Hay que notar que a ser errores anteriores no hubo cambios, lo cual mantiene consistente al sistema.

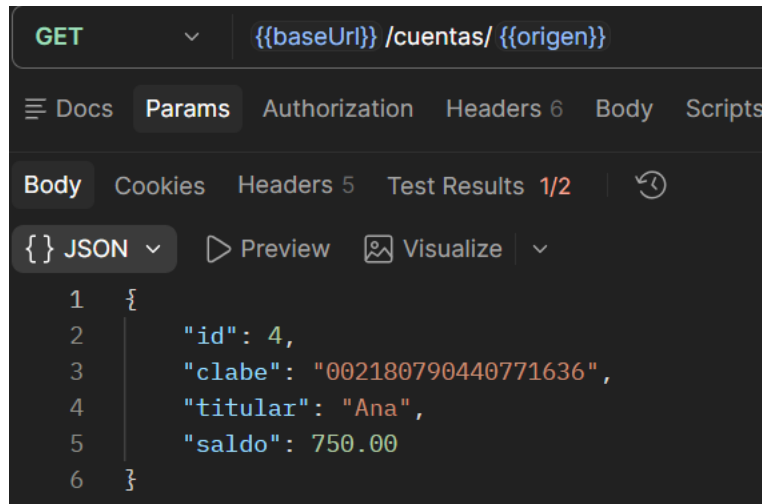


Paso 4. Bájaló con y sin volumen

Cuando Docker Compose down, podemos checar el salud desde Postman y podemos observar que no hay conexión con el localhost.



Posteriormente con Docker Compose up, podemos consultar los mismo datos aun con al transacciones ya realizadas.

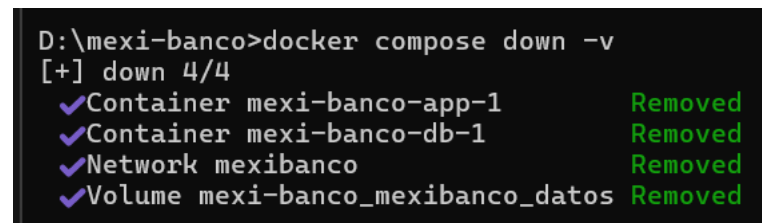


```
GET {{baseUrl}} /cuentas/ {{origen}}

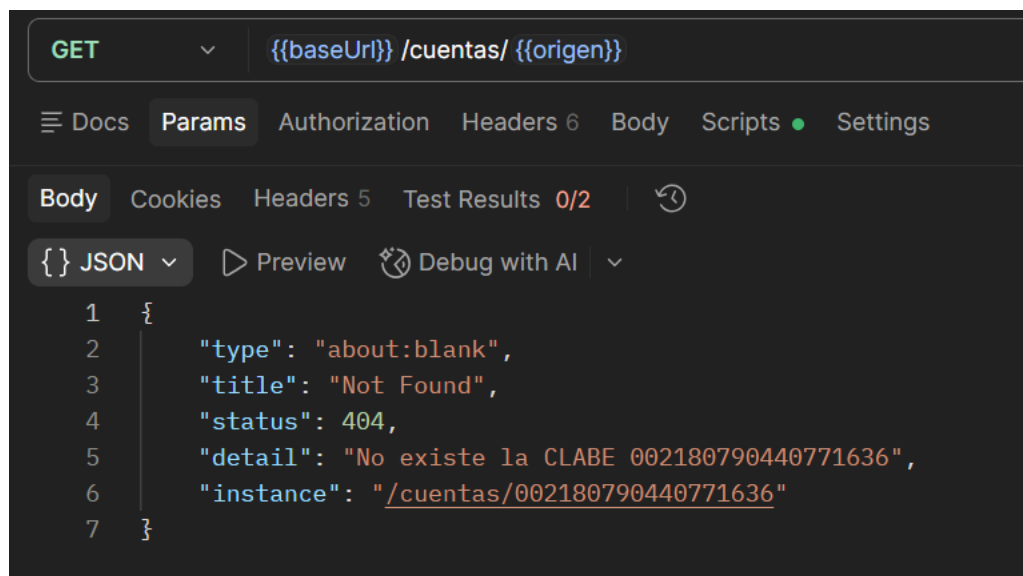
Body
  Cookies
  Headers 5
  Test Results 1/2
  { } JSON
  Preview
  Visualize

1 {
2   "id": 4,
3   "clabe": "002180790440771636",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

Si, eliminamos el contenedor y sus volumen, ocurre un problema a la información se pierde y no podemos consultarla, como aparece en error que no la CLABE no existe.



```
D:\mexi-banco>docker compose down -v
[+] down 4/4
✓Container mexi-banco-app-1      Removed
✓Container mexi-banco-db-1      Removed
✓Network mexibanco              Removed
✓Volume mexi-banco_mexibanco_datos Removed
```

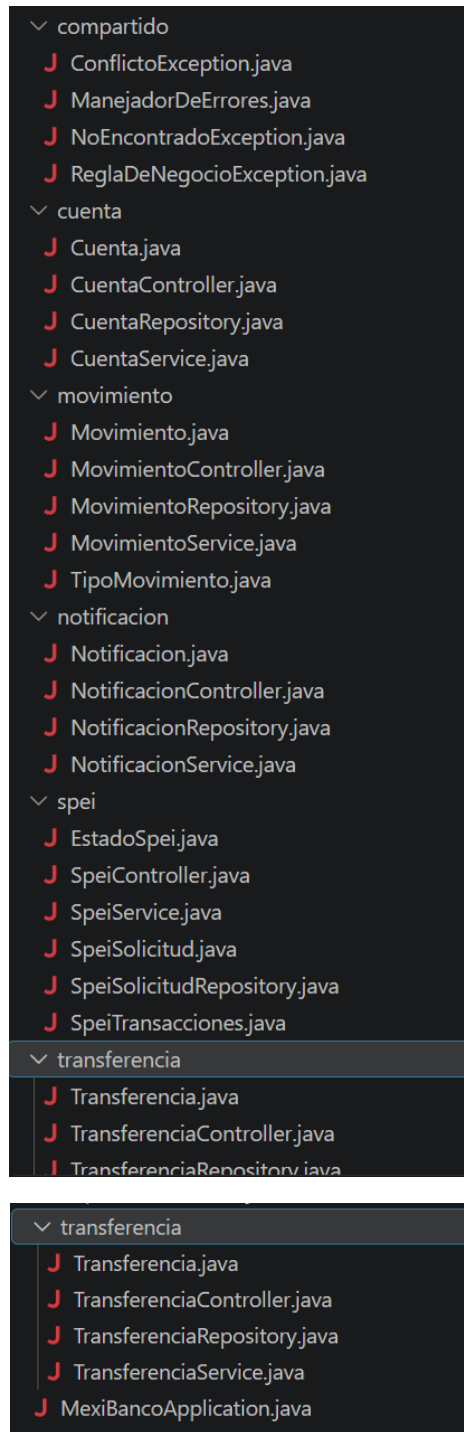


```
GET {{baseUrl}} /cuentas/ {{origen}}

Body
  Cookies
  Headers 5
  Test Results 0/2
  { } JSON
  Preview
  Debug with AI

1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 002180790440771636",
6   "instance": "/cuentas/002180790440771636"
7 }
```

Paso 5. Mexi-Banco



- **Compartido:** Agrupa clases transversales y de uso general, como excepciones personalizadas y el manejo centralizado de errores.

- **Cuenta:** Contiene los datos y la lógica de las cuentas bancarias (entidad, controlador, repositorio y servicio).
- **Movimiento:** Administra los registros de transacciones financieras individuales mediante su entidad, controlador, repositorio, servicio y tipos de movimiento.
- **Notificacion:** Gestiona el envío de avisos y alertas del sistema con su respectiva estructura modular.
- **Spei:** Integra los componentes para procesar los pagos electrónicos, abarcando estados, solicitudes, transacciones, repositorio, servicio y controlador.
- **Transferencia:** Maneja las operaciones de transferencia de fondos entre cuentas a través de su entidad, controlador y repositorio.

- **Paso 6. Docker y Docker-compose**

```
# ---- ETAPA 1: CONSTRUCCION ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN apk add dos2unix && dos2unix ./mvnw && chmod +x ./mvnw && ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests

# ---- ETAPA 2: EJECUCION ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Este Dockerfile utiliza una construcción en dos etapas (multi-stage build) para empaquetar la aplicación de manera eficiente: en la primera etapa (builder), toma el código fuente, descarga las dependencias y compila el proyecto usando Maven con Java 21 para generar el archivo ejecutable (.jar); luego, en la segunda etapa, toma únicamente ese archivo empaquetado y lo coloca en una imagen ligera de Java optimizada para producción, exponiendo el puerto 8080 y configurando el comando de inicio para ejecutar la aplicación.

```
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: mexibanco
      POSTGRES_USER: mexibanco
      POSTGRES_PASSWORD: mexibanco
    volumes:
      - mexibanco_datos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: mexibanco
      DB_USER: mexibanco
      DB_PASSWORD: mexibanco
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 5

networks:
  default:
    name: mexibanco

volumes:
  mexibanco_datos:
```

El archivo comienza definiendo la sección `services:` para estructurar los dos contenedores principales del sistema que trabajarán en conjunto. Dentro de esta, el bloque `db:` descarga y levanta la base de datos PostgreSQL utilizando la imagen `postgres:17-alpine`, configurando sus credenciales de acceso en la sección `environment:` y asignando un espacio de almacenamiento seguro con `volumes:` - `mexibanco_datos:/var/lib/postgresql/data` para garantizar que la información persista aunque se apague el contenedor; asimismo, incluye un apartado de `healthcheck:` con el comando `pg_isready` para verificar constantemente que el motor de base de datos esté disponible y listo para recibir conexiones.

Por otro lado, el bloque `app:` utiliza la instrucción `build:` para compilar la aplicación a partir del código fuente y abre el acceso exterior mediante `ports:` - `"8080:8080"`. En su

configuración de `environment`:, especifica los datos de conexión apuntando al servicio de base de datos (`DB_HOST: db`), vinculándose estrictamente mediante `depends_on`: con la condición `service_healthy` para asegurar que la aplicación no arranque hasta que PostgreSQL esté completamente operativo, y añade su propio `healthcheck`: consultando la ruta web `/actuator/health`.

Finalmente, las últimas secciones del archivo configuran `networks`: para aislar y conectar ambos servicios bajo una red privada común llamada `mexibanco`, facilitando su comunicación interna de manera segura, y cierran declarando el volumen `mexibanco_datos` para habilitar el almacenamiento persistente de la base de datos.